NAME

read_pin_constraints
Reads pin constraints from the specified file and applies them to the current design.

SYNTAX

```
status read_pin_constraints
[-file_name file_name]
```

Data Types

file_name string

ARGUMENTS

-file_name name
Specifies the file name that contains the pin constraints. The constraints are used by the router and pin placer.

DESCRIPTION

This command reads a file which contains pin and feedthrough constraints. The pin constraint file contains multiple sections that contain different types of constraints for the router and pin placer. Four sections are supported: topological map, physical pin constraints, pin spacing control, block pin constraints. Pin-based physical pin constraints supports multilevel physical hierarchy (MPH).

This command stores the pin constraints in the database. The constraints are treated the same as constraints that are set with other Tcl commands. You can use the report_individual_pin_constraints and remove_individual_pin_constraints commands to report and remove pin constraints.

The set_editability command can enable or disable this command for blocks or levels of physical hierarchy. This command has no effect on the blocks that are disabled with the set_editability command.

Each section begins with a section header. Supported section headers are "start topological map", "start feedthrough control", "start physical pin constraints", "start pin spacing control", "start block pin constraints;". Each section ends with a section footer and a semicolon (;). The section footer corresponds to the section header and begins with the "end" keyword, for example "end topological map;". The constraint file specifies how nets should be routed through blocks in routing stage and how pin should be placed during pin placement.

Empty lines or lines with only white spaces are ignored. Leading or trailing white spaces are ignored. Lines that begin with comment character (#) are comments and are ignored.

Curly brackets ({}) are used to separate different constraints and objects. A semicolon (;) separates different records. If there is more than one object following a constraint keyword, you must use curly brackets. In the following example, the first layer constraint is incorrect because the layer names are not surrounded by curly brackets. The second constraint shows the correct syntax.

```
{{cell A} {layers M2 M4}}  # incorrect
{{cell A} {layers {M2 M4}} # correct
```

Topological Map

The topological map section contains routing topologies for different nets in the design. Add constraints to this section to specify how nets should be routed through blocks during pin placement.

Each record contains one or more net or bundle names, followed by pairs of linked objects. Use curly brackets to specify more than one net name. The objects can be block cell, top-level port or any cell instance pin. The pin can be a macro pin or standard cell pin, but cannot be the pin on physical block. Note that ports and pins cannot be specified for bundles. In following example, a net connects to cell_A, Cell_C, and Cell_E, you can force the routing to go through Cell_B to connect Cell_A and Cell_C as follows:

```
start topological map;
{Nets Net_A}
  {{{Cell Cell_A} {Cell Cell_B}}
```

```
   {{Cell Cell_B} {Cell Cell_C}}
   {{Cell Cell_A} {Cell Cell_E}}};
end topological map;
```

The following example forces routing to pass through Cell_B to connect top-level port A with a macro or standard cell pin C/a if Net_A does not connect to Cell_B:

```
start topological map;
{Nets Net_A}
  {{{Port A} {Cell Cell_B}}
   {{Cell Cell_B} {Pins C/a}}};
end topological map;
```

When you have a set of nets with the same routing patterns, you can describe them as a group of nets within curly brackets separated by spaces. In addition, you can use "*" as a wildcard to specify a set of nets, such as a bus. Note that when a set of nets are specified, you cannot use top-level ports or pins to describe the connectivity.

```
start topological map;
{Nets {Net_A Net_B DataIn*}}
  {{{Cell Cell_A} {Cell Cell_B}}
   {{Cell Cell_B} {Cell Cell_C}}};
end topological map;
```

For block objects, you can specify side, offset, and layer constraints for the block pins. The side specifies the side number for a block instance where the pins are inserted. The side number is a positive integer that starts from 1. Given any block instance with a rectangular or rectilinear shape, the leftmost edge is side number 1. If there are multiple leftmost edges, then edge 1 is the lowest leftmost edge. The sides are numbered in consecutive order proceeded clockwise around the shape. The shape here referring to block instance's shape, not the reference block's shape.

The offset specifies the distance in microns or percentage from the starting point of a specific edge to the routing cross-point (or the block pin's center location) on that edge in clockwise direction. To specify a percentage, append a '%' character to the end of the float value. Percentage can be a value from -100 to 100 only. The starting point is also following the clockwise fashion. For instance, for a bottom horizontal edge, the starting point is the right vertex of the edge as it is counted in clockwise and bottom edge's starting point is right point vertex. Since the offset is specific on an edge, you must specify the side information with your offset. The edge here is referring to the edge on block instance, not the reference block.The topological constraints not support negative offset value.

The layer constraint specifies the metal layers to use for the pins.

The following example shows one net record in the topological map section that contains side, offset, and layers of the pins. The routing connects Cell_A through side 1 at offset 20.18 on layer M2 to Cell_B through side 3 at offset 40.28 on layer M4. The routing also connects Cell_B through side 1 at offset 30.28 on layer M4 to Cell_C through side 3 at offset 15.88 on layer M2.

```
start topological map;
{Nets Net_A}
   {{{Cell Cell_A} {sides 1}
     {offset 20.18} {layers M2}}
    {{Cell Cell_B} {sides 3}
     {offset 40.28} {layers M4}}}

   {{{Cell Cell_B} {sides 1}
     {offset 30.28} {layers M4}}
    {{Cell Cell_C} {sides 3}
     {offset 15.88} {layers M2}}};
end topological map;
```

The following example shows one net record in the topological map section that contains side, offset, and layers of the pins. The routing connects Cell_A through side 1 at offset 20.18% on layer M2 to Cell_B through side 3 at offset 40.28% on layer M4. The routing also connects Cell_B through side 1 at offset 30.28% on layer M4 to Cell_C through side 3 at offset 15.88% on layer M2.

```
start topological map;
{Nets Net_A}
   {{{Cell Cell_A} {sides 1}
     {offset 20.18%} {layers M2}}
```

```
        {{Cell Cell_B} {sides 3}
           {{{Cell Cell_B} {sides 1}
             {offset 30.28%} {layers M4}}
            {{Cell Cell_C} {sides 3}
             {offset 15.88%} {layers M2}}};
    end topological map;
```

In addition, for block objects, you can specify routing cross-point's location range (or block pin's location range) on a side. Range can be distance or percentage. To specify a percentage, append a '%' character to the end of the float value. Percentage can be a value from -100 to 100 only. The location range is specified with a pair of numbers within {}.

In following example, the routing routes from Cell_A through side 2 with location range between 10 micron offset and 20 micron offset when connecting to Cell_B, and routing routes through Cell_B's side 4 with location range between 10 micron
offset and 20 micron offset.

```
    start topological map;
    {Nets Net_A}
       {{Cell Cell_A} {sides 2}
         {offset {10 20}}}
       {{Cell Cell_B} {sides 4}
         {offset {10 20}}};
    end topological map;
```

In following example, the routing routes from Cell_A through side 2 with location range between 10% offset and 20% offset when connecting to Cell_B, and routing routes through Cell_B's side 4 with location range between 10% offset and 20% offset.

```
    start topological map;
    {Nets Net_A}
       {{Cell Cell_A} {sides 2}
         {offset {10% 20%}}}
       {{Cell Cell_B} {sides 4}
         {offset {10% 20%}}};
    end topological map;
```

For bundle-based topological constraints, the wildcard character (*) is not supported. For instance, you cannot specify a wildcard character for bundles, ports or pins as the bundle is a group of nets. In this context, the wildcard character for bundles, ports or pins can be confusing.

For a very complicated routing topology which involves multiple enters or exits on same block instance, the tool might not be able to generate routing topology that you intend to. In this case, it is recommended to start from the driver to define the routing topology. In the following example, a routing topology started from driver block A and goes to block B with side 1/offset 20, and then goes from block B to load block C. On the separated routing path on same net, the routing branch out from driver block A and goes to block B again and then goes to load block D. With different **offset(40)** on the same **side(1)** of block B, the topological constraints can be specified in following ways:

```
    start topological map;
    {Nets Net_E}
       {{Cell A} {{Cell B} {sides 1}{offset 20}}}
       {{{Cell B} {sides 3}{offset 40}} {Cell C}}
       {{Cell A} {{Cell B} {sides 1}{offset 40}}}
       {{{Cell B} {sides 3}{offset 20}} {Cell D}}
    end topological map;
```

When there are multiple enters or exits on block B to avoid ambiguity, it is recommended to define the topology starting from driver. Also the physical constraints (side or offset or offset range} can be used to help resolve the ambiguity.

Feedthrough Control

The feedthrough control section enables or disables feedthroughs for specific nets and blocks. Each record begins with the Nets keyword and contains one or more net names within curly brackets. The net names are followed by "Feedthrough" or "NoFeedthrough" keywords, followed by block cells within curly brackets.

The following rules are used when applying the feedthrough control:

o Later statements override earlier statements for the same nets.

o The Feedthrough and NoFeedthrough keywords specify whether feedthroughs are allowed or not by default, for either specified block cells or for the entire design. If block cells are specified, all other blocks cells assume the opposite feedthrough constraint value. Therefore, only one Feedthrough or NoFeedthrough constraint line is required per specification, there is no need to specify both the Feedthrough and NoFeedthrough keywords in a single feedthrough specification.

o Topological map constraints can overrule feedthrough control statements. For example, the following topological map constraint forces a feedthrough on block B, regardless of whether feedthroughs are allowed on B.
{{{cell A} {cell B}} {{cell B} {cell C}}}

The following example enables feedthroughs for net Xecutng_Instrn[0] on blocks I_ALU, I_REG_FILE and I_STACK_TOP.

```
start feedthrough control;
{Nets Xecutng_Instrn[0]}
  {Feedthrough {I_ALU I_REG_FILE I_STACK_TOP}};
end feedthrough control;
```

The following example prevents feedthroughs for net Xecutng_Instrn[0] on blocks I_ALU, I_REG_FILE and I_STACK_TOP.

```
start feedthrough control;
{Nets Xecutng_Instrn[0]}
  {NoFeedthrough I_ALU I_REG_FILE I_STACK_TOP};
end feedthrough control;
```

A single Feedthrough or NoFeedthrough statement completely specifies the allowed feedthrough modules for the net. Feedthrough statements are not cumulative. If multiple feedthrough statements exist for the same net, the latest feedthrough statement take over the previous statement. This rule simplifies the organization of the feedthrough file and prevents complex interactions between feedthrough statements.

Feedthrough and NoFeedthrough statements override the default feedthrough permission set by the create_block_pin_constraint -allow_feedthroughs true|false command for the specified nets. However, feedthrough allows or disallows set by set_individual_pin_constraints -allow_feedthroughs true|false command for individual net overrules the feedthrough or NoFeedthrough statements.

The statement "{ Feedthrough A B }" does not force the creation of feedthroughs on A and B. Instead, the statement allows feedthroughs on blocks A and B, while restricting feedthroughs on all other blocks. To force a specific feedthrough path, you must use the pairwise statements described later.

If there are feedthrough constraint conflicts between topological map constraint and previous feedthrough constraints, the feedthrough constraints from constraint file take priority. If there are conflicts between feedthrough constraints and topological map constraints, topological map constraints take priority.

Physical Pin Constraints

The physical pin constraints section specifies physical pin constraints for individual nets, pins or ports. Supported keywords are: net, pins, reference, sides, layers, offset, order, start, end, off_edge, location, width and length.

The syntax for most supported constraint keywords (sides, layers, width, length, and so on) is the same as in the command set_individual_pin_constraints. See the related man pages for more details. For offset, read_pin_constraints can take a number as well as a range, which includes a startpoint and an endpoint.

Each record in the physical pin constraints section is a set of keywords with values enclosed by curly brackets and ended by a semicolon. The first element in the record can be a top-level net or a port for a reference block. If the record is a port object, the block is expected to follow with reference block's name associated with the port.

The physical pin constraints section also supports order for a collection of pins or ports, which is not very convenient by using command set_individual_pin_constraints. To set the order for a collection of pins or ports, it must specify the required edge. And it is also a good practice to specify the range by using start and end. Otherwise, it means the whole edge. In the most simple case as shown in the fol-

lowing example, the distance between the adjacent order constrained pins or ports is roughly equal to the range divided by the number of order constrained pins or ports. Therefore, if there is a big blockage taking away a large chunk of edge, the range should reflect the existence of such a blockage. Generally, the order constraints are intended for a collection of pins or ports on a region with continuous available empty wire tracks for pin placement.

If a specific order plus an exact spacing are required for a set of pins, the recommended approach is to define a bundle and constrain the pin order and spacing with set_bundle_pin_constraints. See man pages of create_bundle and set_bundle_pin_constraints.

The following example applies constraints on a design with two block instances Inst1 and Inst2, whose reference block names are BlockA and BlockB, respectively.

The first two records constrain pins on net E connecting block instances Inst1 and Inst2. The next three records constrain the pins C, D and F on reference block BlockA. The last record set the order of pins in[1], in[2] and in[3] on reference block BlockB.

```
start physical pin constraints;
{Net E} {cell Inst1} {layers METAL2} {sides 2};
{Net E} {cell Inst2} {layers METAL4} {sides 4};
{Pins C} {reference BlockA} {layers METAL2} {sides 1} {offset {40 50}};
{Pins D} {reference BlockA} {layers METAL2} {width 0.8} {length 0.8};
{Pins F} {reference BlockA} {off_edge} {location {10 10}};
{reference BlockB} {sides 2} {start 10} {end 80} {order {in[1] in[2] in[3]}};
end physical pin constraints;
```

The side constraint specifies the side number on which the pin should be inserted for the specified reference block. The side number is a positive integer that starts from one. Given any rectangular or rectilinear shape, the leftmost edge is side number one. If there are multiple leftmost edges, then edge one is the lowest leftmost edge. The sides are numbered in consecutive order proceeded clockwise around the shape. The shape is referring to the reference block's boundary shape.

The offset constraint specifies the distance in microns or percentage from the starting point(positive value) or ending point (negative value) of a specific edge to the routing cross-point on that edge in the clockwise direction. To specify a percentage, append a '%' character to the end of the float value. Percentage can be a value from -100 to 100 only. The starting point is also following the clockwise fashion. For instance, for a bottom horizontal edge, the starting point is the right vertex of the edge as it is counted in clockwise and bottom edge's starting point is right point vertex, vice versa, for ending point is the left vertex of the edge. Since the offset is specific on an edge, you must specify the side information with your offset. The edge here is referring to the reference block's boundary edge.

The location constraints value are referred to the reference cell in the pin constraint entry and therefore, the coordinates are translated to top-level view.

A more complicated order constraint is shown in the following example. The pin order A and B is specified as to be 2 wire tracks in between with A first and B second. B also must use layer M4. Pins A, B, C and D must be placed within the region between 20 to 40 microns on side 3 of block RISC_CORE. Because pin C and D are not specified within the {order...} constraint, they can be anywhere within the 20 to 40 micron region on side 3.

```
start physical pin constraints;
{reference RISC_CORE}
  {sides 3} {start 20}
  {end 40} {{order {A {spacing 2} {layers METAL4} B}} {pins {C D}}};
end physical pin constraints;
```

If there are multiple records constraining the same object with conflict settings, the last record overwrites the previous ones.

Finally, pin constraints can be applied by either one of the commands set_individual_pin_constraints, set_bundle_pin_constraints, or read_pin_constraints. But try to use the same method to apply the constraints if possible so as to avoid unwanted conflict between different methods. If unclear whether or not the database was applied constraints before, or the type of the constraints applied, use remove_individual_pin_constraints or remove_bundle_pin_constraints to clear them.

Pin Spacing Control

The pin spacing control section specifies pin spacing constraints for individual edges on each block cell.

Each record specifies the pin spacing constraints for one or more edges on one or more layers for a particular block cell. The constraint contains the reference, side, and layer keywords. Keywords are case insensitive.

In addition, for those edges specified in the pin spacing control section, those layers that are NOT listed are NOT allowed for pin placement. Furthermore, if those listed layers are NOT allowed in the corresponding reference blocks, they are NOT allowed for pin placement either.

```
start pin spacing control;
{reference ref_cell_name} {sides {side_a side_b}}
{layer_a spacing_a} {layer_b spacing_b};
end pin spacing control;
```

The reference keyword specifies the block reference that this constraint applies to. For a block cell, this constraint refers to the reference cell name, not the cell instance name, if it is to constrain the top level.

The sides keyword specifies one or more sides of the block. If there is more than one side, specify the side number within curly brackets separated by white space. You can use a wildcard character (*) to specify that all sides are allowed.

After the side constraint, layer and spacing constraints are specified as pairs. You can specify multiple layer-spacing pairs. The layer value is the layer name. The spacing value is the minimum number of wire tracks between adjacent pins for the specified layer.

In the following example, the CPU block cell is constrained to minimum pin spacing of 2 wire tracks on layer M2, a minimum pin spacing of 3 wire tracks on layer M4, and minimum pin spacing of 3 wire tracks on layer M6 for sides 2 and 4 of the block:

```
start pin spacing control;
{reference CPU} {sides 2 4} {METAL2 2} {METAL4 3} {METAL6 3};
end pin spacing control;
```

Keywords are case insensitive. The reference name and the layer name are case-sensitive.

If a line contains a syntax error, the line is ignored. The command issues warnings or errors that report any lines that are ignored.

If the pin spacing control section contains conflicting constraints for the same edge, same layer and the same block, the last constraint takes priority.

Block Pin Constraints

The block pin constraints section specifies pin constraints that apply to all the pins of the specified block cell.

You can specify block-related pin constraints for feedthroughs, layers, spacing, corner keepouts, hard pin constraints, sides and exclude sides. The same constraint can be set with the create_block_pin_constraint Tcl command. Supported keywords are: reference, sides, exclude_sides, layers, feedthrough, spacing, spacing_distance, corner_keepout_distance, corner_keepout_num_tracks and hard_constraints.

The following example sets pin constraints:

```
start block pin constraints;
{reference DATA_PATH}
  {layers {METAL3 METAL4 METAL5}}
  {feedthrough true}
  {corner_keepout_num_tracks 1}
  {spacing 5}
  {exclude_sides {2 3}};

{reference ALU}
  {layers {METAL3 METAL4 METAL5}}
  {feedthrough true}
  {hard_constraints {spacing location layer}}
  {sides {2 3}};
end block pin constraints;
```

You can use the write_pin_constraints command to write out block cell

pin constraints to a file, and use the read_pin_constraints command to read the file.

Bundle Pin Constraints

The bundle pin constraints section specifies bundle pin-related constraints. This section is now included in physical pin constraints section.

You can specify whether bundle pin constraints are honored or not during pin placement. The same constraints can be set with the set_bundle_pin_constraints -keep_pins_together true|false command. The following example sets bundle pin constraints:

```
start physical pin constraints;
{bundle bundle_0} {keep_bundle_pins_together true};
end physical pin constraints;
```

You can use the write_pin_constraints command to write out the bundle pin constraints to a file, and use the read_pin_constraints command to read the file.

Conflicts

When there is a conflict between a topological map constraint and a physical pin constraint, topological map constraints takes priority over physical pin constraints for global routing and pin placement.

The constraint file can have multiple topological map sections or multiple physical pin constraints sections. If same object is listed multiple times with the same constraint, the last record takes priority. If the same object is listed multiple times with different constraints, the constraints are combined.

EXAMPLES

The following example reads the pin_cstr pin constraint file.

```
prompt> read_pin_constraints -file_name pin_cstr
```

A complete pin constraints file example is shown as follows:

```
start topological map;
{nets net_B}
{{{cell Cell_A} {sides 1}
  {offset 20.18} {layers M2}}
 {{cell  Cell_B} {sides 3}
  {offset 40.28} {layers M4}}
 {{cell  Cell_C} {sides 1}
  {offset 30.28} {layers M4}}
 {{cell  Cell_D} {sides 3}
  {offset 15.88} {layers M2}}};
end topological map;

start feedthrough control;
{nets Xecutng_Instrn[0]}
  {NoFeedthrough {I_ALU I_REG_FILE I_STACK_TOP}};
end feedthrough control;

start physical pin constraints;
  {pins C} {reference B} {layers METAL2 METAL4} {sides 2};
  {pins D} {reference B} {sides 2};
  {net C/A} {layer METAL4};
  {bundle bundle_0} {keep_bundle_pins_together true}
end physical pin constraints;

start pin spacing control;
{reference C} {sides 2 4} {METAL2 2} {METAL4 3} {METAL6 3};
end pin spacing control;

start block pin constraints;
{reference DATA_PATH}
  {layers {METAL3 METAL4 METAL5}}
  {feedthrough true}
  {corner_keepout_num_tracks 1}
  {spacing 5}
  {exclude_sides {2 3}};

{reference ALU}
  {layers {METAL3 METAL4 METAL5}}
  {feedthrough true}
  {hard_constraints {spacing location layer}}
  {sides {2 3}};
end block pin constraints;
```

SEE ALSO

remove_individual_pin_constraints(2)

report_individual_pin_constraints(2)
set_individual_pin_constraints(2)
set_editability(2)
